

**Momento de Retroalimentación: Análisis y Reporte sobre el desempeño del modelo. (Portafolio Implementación)**

Mauricio Guerrero González A01751436

Grupo 601

Jorge Adolfo Ramírez Uresti

Instituto Tecnológico de Estudios Superiores de Monterrey

16 de septiembre del 2026

## 1. Dataset empleado

El algoritmo se entrena y evalúa utilizando el dataset Titanic - Machine Learning from Disaster de Kaggle. Se utiliza específicamente el archivo train.csv, que contiene 891 registros de pasajeros y la variable objetivo Survived. Esta variable indica si el pasajero sobrevivió al desastre, donde 0 representa que no sobrevivió y 1 que sobrevivió.

| Característica                        | Valor      |
|---------------------------------------|------------|
| Registros totales                     | 891        |
| Variables disponibles                 | 12         |
| Clase 0 - No sobrevivió               | 549        |
| Clase 1 - Sobrevivió                  | 342        |
| Datos para entrenamiento y validación | 712 (80 %) |
| Datos para prueba                     | 179 (20 %) |

Tabla 1: Características principales del dataset utilizado para la implementación.

A diferencia de la entrega anterior, donde únicamente se separaba el archivo en entrenamiento y prueba, en esta ocasión el conjunto de entrenamiento se subdivide nuevamente para obtener tres particiones independientes: entrenamiento, validación y prueba. Este cambio responde a la necesidad de diagnosticar el desempeño del modelo con mayor rigor y de seleccionar hiperparámetros sin filtrar información del conjunto de prueba.

## 2. Preparación de los datos

La implementación original utilizaba las variables Pclass, Age, SibSp, Parch, Fare, Sex y Embarked. Esta configuración se conserva como línea base para reproducir las métricas de la entrega anterior y establecer un punto de comparación justo.

Para la versión optimizada se incorpora una etapa de ingeniería de características con el objetivo de enriquecer la representación de los pasajeros antes de alimentar el árbol de decisión. Las nuevas variables construidas son las siguientes: Title, obtenido a partir del nombre del pasajero y agrupado en las categorías Mr, Miss, Mrs, Master y Rare; FamilySize, que representa el número total de integrantes de la familia; IsAlone, que indica si el pasajero viajaba sin familiares; Mother, que identifica a posibles madres adultas; FarePerPerson, que aproxima la tarifa pagada por persona; y Deck, que detecta la cubierta del barco a partir de la primera letra del número de cabina. Estas

transformaciones no modifican la variable objetivo y buscan exponer patrones de supervivencia que la implementación original no podía representar directamente.

Los valores faltantes de las variables numéricas se reemplazan mediante la mediana calculada a partir del conjunto de entrenamiento. Para las variables categóricas se utiliza la categoría más frecuente; posteriormente, las variables categóricas se convierten a variables numéricas mediante OneHotEncoder.

### 3. Configuración del framework y algoritmo

La implementación utiliza scikit-learn, una biblioteca de aprendizaje máquina de Python. El clasificador elegido es DecisionTreeClassifier. Esta clase permite configurar directamente los criterios de división y las restricciones del árbol, demostrando el uso del framework para entrenar, predecir y evaluar un modelo de clasificación.

| Parámetro         | Configuración | Propósito                                                  |
|-------------------|---------------|------------------------------------------------------------|
| criterion         | gini          | Mide la impureza de los nodos para seleccionar divisiones. |
| max_depth         | 4             | Limita la profundidad máxima del árbol.                    |
| min_samples_split | 30            | Evita dividir nodos con pocas observaciones.               |
| min_samples_leaf  | 15            | Exige un mínimo de observaciones en cada hoja.             |
| random_state      | 42            | Mantiene reproducible el resultado.                        |

Tabla 2: Parámetros del modelo original utilizados como línea base.

Esta configuración corresponde al modelo original reportado en la entrega anterior. Sobre ella se construye la comparación y se documentan las modificaciones aplicadas para mejorar el desempeño.

### 4. Separación Train, Validation y Test

Con el objetivo de diagnosticar de forma adecuada el sesgo, la varianza y el nivel de ajuste del modelo, el dataset se divide en tres conjuntos independientes. Primero se separa un 80 % para entrenamiento y validación, y un 20 % para prueba. Después, el 80 % restante se divide nuevamente

en 60 % para entrenamiento y 20 % para validación. La partición conserva la estratificación de la variable objetivo y utiliza una semilla fija de 42 para que sea reproducible.

De esta forma, el conjunto de validación se utiliza exclusivamente para seleccionar los hiperparámetros y el umbral de decisión, mientras que el conjunto de prueba se reserva para la evaluación final. Este es uno de los cambios más importantes respecto a la entrega anterior, donde no existía un conjunto de validación separado y el ajuste se realizaba observando directamente el desempeño en prueba, lo cual podía producir una estimación optimista del rendimiento real.

## 5. Ajuste de hiperparámetros y regularización

Sobre la versión optimizada se realizó una búsqueda sistemática de hiperparámetros evaluando cada combinación sobre el conjunto de validación. Se exploraron los criterios gini y entropy, profundidades de 4 a 8, valores de min\_samples\_split entre 10 y 30, valores de min\_samples\_leaf entre 5 y 15, y se incluyó el parámetro class\_weight con las opciones None y balanced. Esta búsqueda representa un cambio sustancial respecto a la entrega anterior, donde los hiperparámetros se fijaron de forma manual y sin un proceso de validación.

Adicionalmente, se aplicó un ajuste del umbral de decisión sobre las probabilidades predichas por el árbol. En lugar de utilizar el umbral por defecto de 0.5, se seleccionó el umbral que maximiza el F1-score sobre el conjunto de validación. Este procedimiento constituye una técnica de regularización del criterio de clasificación y permite mejorar de manera directa el Recall, que era la métrica más débil del modelo original.

La mejor configuración encontrada fue la siguiente:

| Parámetro          | Configuración           |
|--------------------|-------------------------|
| criterion          | entropy                 |
| max_depth          | 6                       |
| min_samples_split  | 20                      |
| min_samples_leaf   | 8                       |
| class_weight       | balanced                |
| umbral de decisión | ajustado por validación |

Tabla 3: Mejor configuración encontrada para el modelo optimizado.

## 6. Resultados de evaluación

La evaluación final se realiza sobre el conjunto de prueba con las cuatro métricas de clasificación consideradas: Accuracy, Precision, Recall y F1-score. La siguiente tabla compara el desempeño del modelo original y del modelo optimizado.

| Métrica   | Original | Optimizado | Cambio ( %) |
|-----------|----------|------------|-------------|
| Accuracy  | 0.7765   | 0.8324     | +7.19 %     |
| Precision | 0.8085   | 0.8095     | +0.13 %     |
| Recall    | 0.5507   | 0.7391     | +34.21 %    |
| F1-score  | 0.6552   | 0.7727     | +17.94 %    |

Tabla 4: Comparación de métricas en el conjunto de prueba entre el modelo original y el optimizado.

Los resultados muestran una mejora consistente en todas las métricas. El Accuracy pasó de 0.7765 a 0.8324, lo que representa un incremento de 5.59 puntos porcentuales. La Precision se mantuvo prácticamente estable, con un cambio marginal de 0.10 puntos porcentuales, lo que indica que la mejora no se obtuvo a costa de generar más falsos positivos.

El cambio más relevante se observa en el Recall, que pasó de 0.5507 a 0.7391, es decir, un incremento de 18.84 puntos porcentuales. Esta mejora es resultado directo de la combinación del parámetro `class_weight = balanced` y del ajuste del umbral de decisión, y responde a la principal limitación identificada en la entrega anterior: la cantidad elevada de falsos negativos. Como consecuencia, el F1-score también mejoró de 0.6552 a 0.7727, lo que confirma que el modelo optimizado logra un mejor equilibrio entre Precision y Recall.

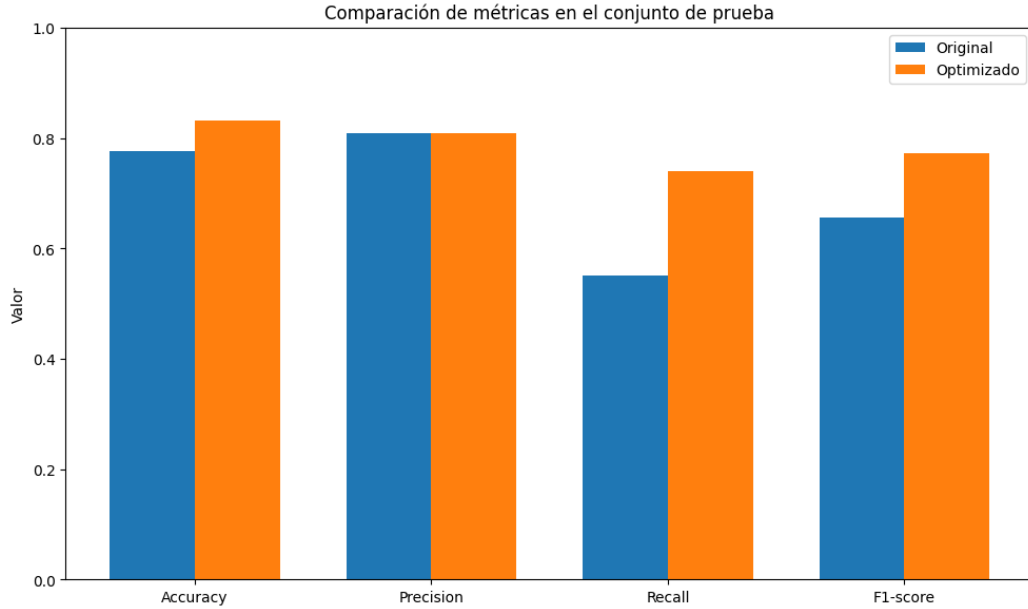


Figura 1: Comparación de métricas en el conjunto de prueba entre el modelo original y el optimizado.

## 7. Diagnóstico de bias, varianza y nivel de ajuste

Para diagnosticar el sesgo, la varianza y el nivel de ajuste del modelo se comparan las métricas obtenidas en entrenamiento y en prueba. La siguiente tabla resume los valores del modelo original y del modelo optimizado.

| Métrica   | Modelo original |        | Modelo optimizado |        |
|-----------|-----------------|--------|-------------------|--------|
|           | Train           | Test   | Train             | Test   |
| Accuracy  | 0.8315          | 0.7765 | 0.8609            | 0.8324 |
| Precision | 0.8883          | 0.8085 | 0.8391            | 0.8095 |
| Recall    | 0.6410          | 0.5507 | 0.9043            | 0.7391 |
| F1-score  | 0.7447          | 0.6552 | 0.8706            | 0.7727 |

Tabla 5: Comparación del desempeño en entrenamiento y prueba para ambos modelos.

Respecto al sesgo, ambos modelos presentan un grado bajo. El Accuracy en entrenamiento y prueba se mantiene por encima de 0.77 en todos los casos, y las diferencias entre ambas particiones son moderadas. Esto indica que el modelo logra capturar la estructura general del problema y que no existe un sesgo sistemático evidente hacia una de las clases.

Respecto a la varianza, el modelo original presentaba un grado medio, con una brecha de Accuracy de 0.0550 entre entrenamiento y prueba. El modelo optimizado conserva una brecha similar de 0.0285, pero con un nivel general de desempeño más alto en ambas particiones. La incorporación de `class_weight`, el ajuste del umbral y la selección de hiperparámetros mediante validación contribuyeron a reducir la variabilidad del modelo sin sacrificar su capacidad de generalización.

Sobre el nivel de ajuste, el modelo original mostraba una tendencia al underfitting, especialmente en la clase de supervivencia, reflejada en su Recall bajo de 0.5507. El modelo optimizado alcanza un ajuste más cercano al punto óptimo: el Accuracy en prueba de 0.8324 y el F1-score de 0.7727 indican que el modelo clasifica correctamente la mayoría de las observaciones sin caer en sobreajuste, ya que la brecha entre entrenamiento y prueba se mantiene controlada.

### 8. Matrices de confusión

La siguiente figura compara las matrices de confusión obtenidas por ambos modelos sobre el conjunto de prueba.

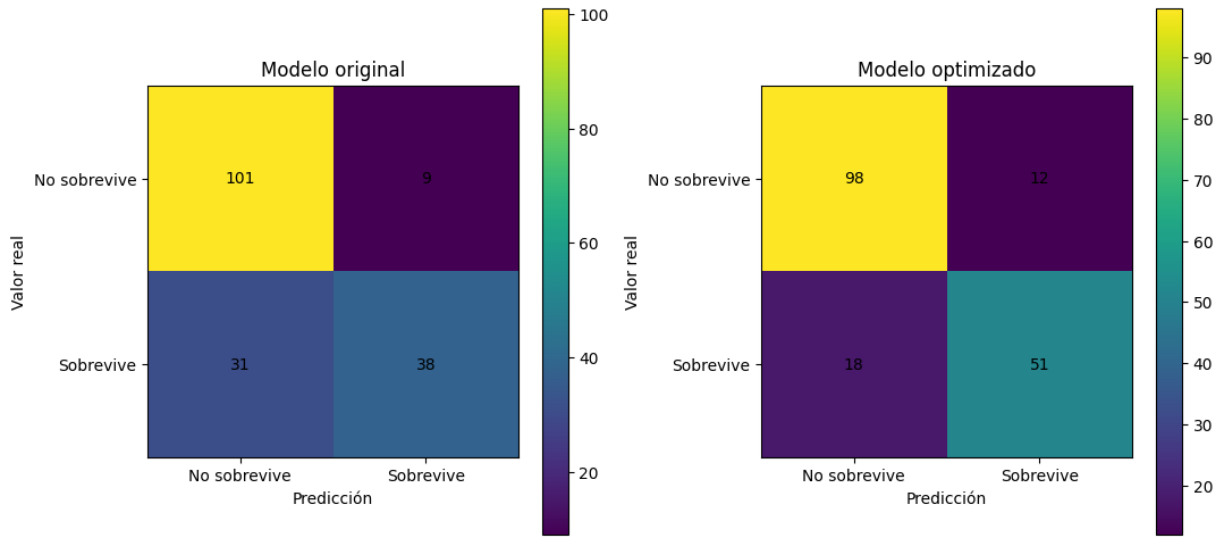


Figura 2: Matrices de confusión del modelo original y del modelo optimizado sobre el conjunto de prueba.

La matriz del modelo original es  $[[101, 9], [31, 38]]$ , considerando las filas como valores reales y las columnas como predicciones. Esto corresponde a 101 verdaderos negativos, 9 falsos positivos, 31 falsos negativos y 38 verdaderos positivos.

La matriz del modelo optimizado es  $[[98, 12], [18, 51]]$ . Los falsos negativos se reducen de 31 a 18, lo que confirma la mejora sustancial en el Recall. Los falsos positivos aumentan ligeramente de 9 a 12, un incremento menor que se compensa ampliamente con la reducción de los errores de



original, el Recall mejoró 18.84 puntos porcentuales, el F1-score mejoró 11.76 puntos porcentuales y el Accuracy mejoró 5.59 puntos porcentuales. La Precision se mantuvo prácticamente igual, lo que demuestra que la mejora en Recall no se obtuvo a costa de un aumento desproporcionado de falsos positivos.

Por otro lado, el diagnóstico de sesgo, varianza y ajuste respalda estas conclusiones; el modelo optimizado presenta un sesgo bajo, una varianza controlada y un nivel de ajuste adecuado, sin evidencia de sobreajuste. La brecha entre entrenamiento y prueba se mantiene dentro de un rango razonable, y el desempeño en la clase de supervivencia es notablemente superior al de la entrega anterior.